

URBAN DATA EXPLORER

Calcul détaillé des 4 indicateurs composites

Bloc 1 — Architecture de données · RNCP40875

Projet	Urban Data Explorer — Analyse immobilière Paris
Certification	RNCP40875 — Expert en Ingénierie des données
Compétences	C2.3 (transformation multi-sources) · C2.4 (optimisation)
Étudiants	Yoann DOSSOU-YOVO · Cheick COULIBALY · Yannick ONGOLO ONGOLO · Darrel NGADJUI
École	Efrei — Paris Panthéon-Assas Université
Soutenance	2 & 3 juillet 2026

Ce document explique en détail comment les 4 indicateurs composites de l'Urban Data Explorer sont calculés, quelles sources de données sont utilisées, comment les données brutes sont normalisées, et comment les pondérations sont appliquées pour produire un score final de 0 à 10 par arrondissement parisien.

Vue d'ensemble du système d'indicateurs

L'Urban Data Explorer produit **4 scores composites** par arrondissement parisien, tous normalisés sur une échelle de **0 à 10**. Ces scores sont calculés dans le script `pipeline/gold_indicateurs.py` à partir des données Silver (Parquet nettoyées) et stockés dans la couche Gold.

Indicateur	Ce qu'il mesure	Fichier Gold
I1 — Accessibilité urbaine	Services et équipements du quotidien	indicateurs_custom.parquet
I2 — Qualité de vie	Environnement naturel et pollution	indicateurs_custom.parquet
I3 — Sécurité	Délinquance et présence sécuritaire	indicateurs_custom.parquet
I4 — Tension immobilière	Effort financier achat / location	indicateurs_custom.parquet
Score global	Moyenne des 4 indicateurs	indicateurs_custom.parquet
Prix m ² médian (bonus)	Prix immobilier brut DVF par année	agregats_arrondissements.parquet

Fonction de normalisation (commune à tous les indicateurs) :

```
normalize_0_10(série, inverse=False) :  
mn = série.min() ; mx = série.max()  
norm = (valeur - mn) / (mx - mn) × 10
```

Si `inverse=True` → retourne $(10 - \text{norm})$

← utilisé quand une valeur haute est mauvaise (ex : criminalité)

■ Si `min = max` (toutes les valeurs identiques), la fonction retourne 5.0 pour tous les arrondissements afin d'éviter une division par zéro.

Chaîne de traitement générale :

```
Sources brutes → Silver Parquet (nettoyées)  
↓ groupby(arrondissement) + agrégation (count / sum / median)  
Valeurs brutes par arrondissement  
↓ normalize_0_10() → sous-scores 0-10  
↓ pondération (× poids)  
Score composite final 0-10  
↓ sauvegarde Parquet Gold + PostgreSQL (UPSERT)
```

1

Accessibilité urbaine

Objectif : mesurer à quel point un arrondissement est bien desservi et équipé au quotidien — transports, commerces, santé, éducation, et position géographique centrale.

Sources et sous-scores pondérés :

Transports (IDFM)	40%	Nombre d'arrêts métro/bus/RER dans l'arrondissement
Commerces (OSM/SIRENE)	20%	nb_supermarches + nb_pharmacies + nb_boulangeries
Santé (FINESS/OSM)	15%	nb_hopitaux + nb_medecins
Éducation (education)	15%	Nombre d'écoles, collèges, lycées
Centralité géo.	10%	Score fixe par position (1er=10, 20e=6, 19e=5.5...)

Détail du calcul :

```
# Données Silver chargées depuis Parquet
df_tr = silver/transports.parquet # IDFM – arrêts par coord. GPS
df_edu = silver/education.parquet # data.education.gouv.fr
df_cs = silver/commerces_sante.parquet # OSM + FINESS/SIRENE

# Agrégation par arrondissement
nb_arrets = df_tr.groupby('arrondissement').size()
nb_ecoles = df_edu.groupby('arrondissement').size()
nb_commerces = df_cs['nb_supermarches'] + df_cs['nb_pharmacies'] +
df_cs['nb_boulangeries']
nb_sante = df_cs['nb_hopitaux'] + df_cs['nb_medecins']

# Normalisation 0-10
s_transport = normalize_0_10(nb_arrets)
s_ecoles = normalize_0_10(nb_ecoles)
s_commerces = normalize_0_10(nb_commerces)
s_sante = normalize_0_10(nb_sante)
centralite = {1:10, 2:9.5, 3:9, 4:9.5, 5:8.5, 6:8.5,
7:8, 8:8, 9:7.5, 10:7, 11:7.5, 12:6.5,
13:6, 14:6, 15:6.5, 16:6, 17:7, 18:6.5,
19:5.5, 20:6}
```

Formule finale :

```
score_accessibilite =
s_transport × 0.40
+ s_commerces × 0.20
+ s_sante × 0.15
+ s_ecoles × 0.15
+ centralite × 0.10
```

Interprétation :

Score	Signification	Exemples attendus
-------	---------------	-------------------

8 – 10	Arrondissement très bien desservi	1er, 2e, 6e, 8e — centre de Paris
5 – 7	Desserte correcte, quelques lacunes	10e, 11e, 15e, 17e
0 – 4	Moins équipé, excentré ou peu de transports	19e, 20e

■ *Fallback* : si un fichier Silver est absent (FINESS indisponible par exemple), la colonne correspondante est mise à 0 pour tous les arrondissements sans bloquer le calcul.

12

Qualité de vie

Objectif : mesurer la qualité de l'environnement de vie au sens large — accès à la nature, qualité de l'air, niveau sonore et fluidité du trafic. Cet indicateur est entièrement basé sur des données environnementales.

Sources et sous-scores pondérés :

Parcs (Overpass OSM)	30%	Nombre d'espaces verts > 500m ² dans l'arrondissement
Qualité de l'air (AIRPARIF)	25%	score_air (0–100 → /10) — polluants PM2.5, NO2, O3
Bruit urbain (BRUITPARIF)	25%	score_bruit (0–10) — niveau Lden en décibels inversé
Circulation (Paris Open Data)	20%	score_circulation (0–10) — intensité trafic inversée

Détail du calcul :

```
# Données Silver chargées
df_parcs = silver/parcs.parquet # OSM — polygones espaces verts
df_air = silver/qualite_air.parquet # AIRPARIF — score_air déjà calculé
df_bruit = silver/bruit.parquet # BRUITPARIF — score_bruit Lden
df_circ = silver/circulation.parquet # Paris OD — score_circulation

# Parcs : compte le nombre de parcs par arrondissement
nb_parcs = df_parcs.groupby('arrondissement').size()
s_parcs = normalize_0_10(nb_parcs)

# Air : score_air est déjà sur 0–100 → on ramène sur 0–10
s_air = (df_air['score_air'] / 10).clip(0, 10)

# Bruit : score_bruit est déjà sur 0–10 (Lden inversé dans Silver)
s_bruit = df_bruit['score_bruit'].fillna(5.0)

# Circulation : score_circulation déjà sur 0–10 (trafic inversé)
s_circ = df_circ['score_circulation'].fillna(5.0)
```

Formule finale :

```
score_qualite_vie =
s_parcs × 0.30
+ s_air × 0.25
+ s_bruit × 0.25
+ s_circ × 0.20
```

Interprétation :

Score	Signification	Exemples attendus
8 – 10	Cadre de vie excellent : nature, air pur, calme	16e (bois Boulogne), 12e (bois Vincennes)
5 – 7	Qualité de vie correcte, quelques nuisances	7e, 5e, 14e

0 – 4	Pollution, bruit élevé, peu d'espaces verts	1er, 2e (très denses), 8e
-------	---	---------------------------

■ *Fallback critique : si AIRPARIF est hors quota API ou BRUITPARIF indisponible, les scores correspondants sont remplacés par 5.0 (valeur neutre) pour ne pas biaiser l'indicateur global. Un log WARN est émis pour chaque fallback activé.*

Objectif : mesurer le niveau de sécurité d'un arrondissement en combinant les faits de délinquance enregistrés (score inversé — moins il y en a, mieux c'est) et la présence de forces de sécurité (commissariats et casernes de pompiers).

Sources et sous-scores pondérés :

Criminalité SSMSI	60%	nb_faits délinquance — INVERSÉ (↑ faits = ↓ score)
Commissariats police (OSM)	25%	Nombre de commissariats de police dans l'arrond.
Casernes pompiers (OSM)	15%	Nombre de casernes de pompiers dans l'arrond.

Détail du calcul :

```
# Données Silver chargées
df_crime = silver/criminalite.parquet # SSMSI data.gouv.fr
df_sec_urbaine = silver/securite_urbaine.parquet # OSM commissariats + pompiers

# ■■ Sous-indicateur 1 : Criminalité (60%) – INVERSÉ ■■
nb_faits = df_crime.groupby('arrondissement')['nb_faits'].sum()
s_criminalite = normalize_0_10(nb_faits, inverse=True)
# ↑ inverse=True : arrond. avec MOINS de faits → score plus HAUT

# ■■ Sous-indicateur 2 : Commissariats (25%) ■■
df_police = df_sec_urbaine[df_sec_urbaine['type'] == 'commissariats']
nb_commissariats = df_police.groupby('arrondissement').size()
s_commissariats = normalize_0_10(nb_commissariats)

# ■■ Sous-indicateur 3 : Pompiers (15%) ■■
df_pompiers = df_sec_urbaine[df_sec_urbaine['type'] == 'pompiers']
nb_casernes = df_pompiers.groupby('arrondissement').size()
s_pompiers = normalize_0_10(nb_casernes)
```

Formule finale :

```
score_securite =
normalize_INVERSE(nb_faits) × 0.60 ← INVERSÉ
+ normalize(nb_commissariats) × 0.25
+ normalize(nb_casernes_pompiers) × 0.15
```

Données de référence criminalité (fallback SSMSI) :

Arrondissement	nb_faits estimés	Arrondissement	nb_faits estimés
1er	3 200	11e	4 200
2e	2 100	12e	2 800
3e	1 800	13e	2 600
4e	2 400	14e	2 200

5e	1 500	15e	2 900
6e	1 600	16e	1 900
7e	1 200 ← min	17e	2 700
8e	5 800 ← max	18e	5 100
9e	3 900	19e	3 800
10e	3 500	20e	3 100

Interprétation :

Score	Signification	Exemples attendus
8 – 10	Arrondissement très sûr, peu de faits enregistrés	7e, 5e, 6e, 3e
5 – 7	Sécurité correcte	12e, 14e, 16e
0 – 4	Forte délinquance enregistrée	8e, 18e, 1er, 9e

14

Tension immobilière

Objectif : mesurer si un habitant peut financièrement se permettre d'acheter ou de louer dans l'arrondissement, en croisant les prix immobiliers DVF avec les revenus médians INSEE et les loyers de référence DRIHL. Un score élevé signifie que l'arrondissement est **accessible** financièrement.

Sources et sous-scores pondérés :

Accessibilité achat (DVF + INSEE)	60%	$m2_par_revenu = \text{revenu_median_annuel} / \text{prix_m2_median}$ — m² achetables par an de salaire
Effort locatif (DRIHL + INSEE)	40%	$\text{effort} = \text{loyer_ref_m2} \times 50 / (\text{revenu_median} / 12)$ — part du revenu pour louer 50m²

Détail du calcul :

```
# Données Silver chargées
df_revenus = silver/revenus.parquet # INSEE Filosofi – revenu_median (€/an)
df_loyers = silver/loyers.parquet # DRIHL – loyer_ref_m2 (€/m²/mois)
df_gold_prix = gold/agregats_arrondissements.parquet # DVF – prix_m2_median

# Dernière année disponible DVF (ex : 2024)
derniere_annee = df_gold_prix['annee'].max()
prix = df_gold_prix[df_gold_prix['annee'] == derniere_annee]

# ■■ Sous-score 1 : Accessibilité achat (60%) ■■
# Combien de m² peut-on acheter avec 1 an de revenu médian ?
m2_par_revenu = revenu_median / prix_m2_median
s_achat = normalize_0_10(m2_par_revenu)
# Plus m2_par_revenu est grand → plus l'arrondissement est accessible

# ■■ Sous-score 2 : Effort locatif (40%) ■■
# Quelle fraction du revenu mensuel faut-il pour louer 50m² ?
effort_locatif = (loyer_ref_m2 × 50) / (revenu_median / 12)
s_effort = normalize_0_10(effort_locatif, inverse=True)
# inverse=True : moins d'effort → meilleur score
```

Formule finale :

```
m2_par_revenu = revenu_median_annuel / prix_m2_median
effort_locatif = loyer_ref_m2 × 50m² / (revenu_median / 12)

score_accessibilite_immo =
normalize(m2_par_revenu) × 0.60
+ normalize_INVERSE(effort_locatif) × 0.40
```

Exemples concrets de calcul :

Arrond.	Prix m² DVF	Revenu INSEE	m²/revenu	Effort loc. 50m²	Score estimé
6e	15 000 €/m²	55 000 €/an	3.7 m²	~47% du revenu	Très bas (0–2)

16e	12 000 €/m ²	45 000 €/an	3.75 m ²	~38% du revenu	Bas (2–4)
15e	10 500 €/m ²	35 000 €/an	3.3 m ²	~36% du revenu	Moyen (4–6)
13e	9 500 €/m ²	28 000 €/an	2.9 m ²	~40% du revenu	Moyen (4–5)
19e	9 000 €/m ²	22 000 €/an	2.4 m ²	~49% du revenu	Bas (2–4)
20e	9 500 €/m ²	24 000 €/an	2.5 m ²	~48% du revenu	Bas (2–4)

■ *Remarque : un revenu élevé dans un arrondissement cher (ex: 6e, 16e) ne garantit pas un bon score — c'est le RATIO prix/revenu qui compte. Un revenu faible dans un arrondissement bon marché peut donner un score similaire.*

Score global & Pipeline de production

Score global (moyenne des 4 indicateurs) :

```
score_global = (I1 + I2 + I3 + I4) / 4  
= (score_accessibilite + score_qualite_vie  
+ score_securite + score_accessibilite_immo) / 4
```

■ Le score global est une moyenne arithmétique simple. Aucune pondération n'est appliquée entre les 4 indicateurs
— chacun pèse 25%.

Pipeline complet de production des indicateurs :

```
# 1. Ingestion Bronze (run_all.py)  
python ingestion/run_all.py  
→ data/bronze/transportes.json  
→ data/bronze/education.json  
→ data/bronze/dvf_2021.csv.gz ... dvf_2024.csv.gz  
→ data/bronze/revenus_insee.csv  
→ ... (13 sources)  
  
# 2. Transformation Silver (silver_*.py)  
python pipeline/run_pipeline.py  
→ data/silver/transportes.parquet # géo-jointure GPS → arrond.  
→ data/silver/education.parquet  
→ data/silver/commerces_sante.parquet  
→ data/silver/parcs.parquet  
→ data/silver/qualite_air.parquet # score_air 0-100  
→ data/silver/bruit.parquet # score_bruit 0-10  
→ data/silver/circulation.parquet # score_circulation 0-10  
→ data/silver/criminalite.parquet # nb_faits par arrond.  
→ data/silver/securite_urbaine.parquet # commissariats + pompiers  
→ data/silver/revenus.parquet # revenu_median par arrond.  
→ data/silver/loyers.parquet # loyer_ref_m2  
  
# 3. Gold DVF (gold_agregats.py) – requis pour I4  
→ data/gold/agregats_arrondissements.parquet  
(prix_m2_median × arrondissement × année 2021-2024)  
  
# 4. Gold Indicateurs (gold_indicateurs.py)  
→ data/gold/indicateurs_custom.parquet  
colonnes : arrondissement, score_accessibilite, score_qualite_vie,  
score_securite, score_accessibilite_immo, score_global,  
nb_arrets, nb_ecoles, nb_commerces, nb_sante,  
nb_parcs, nb_faits, nb_commissariats, nb_casernes,  
revenu_median, m2_par_revenu  
  
# 5. Chargement PostgreSQL (UPSERT)  
→ table : indicateur (arrondissement, categorie, annee, valeur, unite)  
  
# 6. Exposition API  
GET /indicateurs/{arrondissement} → DuckDB query sur Gold Parquet
```

Récapitulatif des fichiers Silver requis par indicateur :

Indicateur	Fichiers Silver requis	Fallback si absent
I1 — Accessibilité	transports.parquet education.parquet commerces_sante.parquet	Colonne mise à 0
I2 — Qualité de vie	parcs.parquet qualite_air.parquet bruit.parquet circulation.parquet	5.0 neutre
I3 — Sécurité	criminalite.parquet securite_urbaine.parquet	Valeurs estimées connues
I4 — Tension immo.	revenus.parquet loyers.parquet gold/agregats.parquet	Erreur si revenus manquants

Les 4 indicateurs sont consultables via l'API FastAPI à l'endpoint GET /indicateurs/{arrondissement} (authentification par API Key requise), et visualisés sur le dashboard interactif MapLibre GL JS à <http://localhost:3000> sous forme de carte choroplèthe avec sélecteur d'indicateur.